AWS Data Lake Automation with Glue for Sales Optimization

End-to-End Project Documentation | *Olist Brazilian E-Commerce Dataset*

AWS Region: eu-north-1
Glue database: sales_optimization_db

1. Project Overview

This project builds an AWS data lake for preparing Brazilian e-commerce data for sales analysis. Amazon S3 stores the data, AWS Glue crawlers discover schemas, AWS Glue ETL jobs transform the data, the Glue Data Catalog stores table metadata, and Amazon Athena runs SQL queries.

Objectives

• Land the original Olist CSV files in Amazon S3.

• Catalog the Raw datasets with AWS Glue.

• Transform nine source datasets into a Silver layer.

• Build a Gold sales analytics dataset from Silver data.

• Query the Gold dataset with Athena.

• Validate row counts, selected null fields, and order-item uniqueness.

• Document the implementation for a data engineering portfolio.

Project status

The core pipeline is recorded as implemented and validated. Remaining portfolio work includes finalizing the architecture diagram, saving the scripts and SQL, preparing a README, optionally publishing to GitHub, and reviewing AWS costs and resource retention.

2. Project Configuration

Setting	Value
AWS Region	eu-north-1 (Europe/Stockholm)
S3 bucket	data-engineer-sales-optimization-rakesh-2026
Glue database	sales_optimization_db
Glue IAM role	AWSGlueServiceRole-OlistSalesOptimization
Raw prefix	s3://data-engineer-sales-optimization-rakesh-2026/raw/
Silver prefix	s3://data-engineer-sales-optimization-rakesh-2026/silver/
Gold prefix	s3://data-engineer-sales-optimization-rakesh-2026/gold/sales/
Athena results	s3://data-engineer-sales-optimization-rakesh-2026/athena-results/
Raw crawler	olist-raw-crawler
Gold ETL job	olist-silver-to-gold-sales
Gold catalog table	sales

Keep the AWS Region consistent when opening S3, Glue, IAM, and related resources. The values above are the project identifiers recorded during implementation.

3. Dataset Download and Preparation

Step 1 — Obtain the Olist dataset

1. Download the Brazilian E-Commerce Public Dataset by Olist from its dataset source.

2. Extract the downloaded archive on your computer.

3. Locate the CSV files for orders, order items, customers, payments, reviews, products, sellers, geolocation, and category translation.

4. Keep the original files unchanged as the source copies for the Raw layer.

Step 2 — Confirm the nine datasets

• orders

• order_items

• customers

• payments

• reviews

• products

• sellers

• geolocation

• category_translation

The project uses these nine datasets. If the source archive uses longer filenames, retain those filenames or use the same filenames that were uploaded to S3.

4. Amazon S3 Setup

Step 3 — Select the AWS Region

1. Sign in to the AWS Management Console.

2. Use the Region selector and choose Europe (Stockholm), code eu-north-1.

3. Open Amazon S3.

Step 4 — Create and verify the project bucket

1. Create the bucket named data-engineer-sales-optimization-rakesh-2026, if it does not already exist.

2. Keep public access blocked unless there is a specific, reviewed reason to change it.

3. Open the bucket and verify that you can create folders and upload objects.

Step 5 — Create the Raw folder structure and upload files

1. Create a top-level folder named raw/.

2. Inside raw/, create one folder for each of the nine datasets.

3. Upload each CSV to its matching folder. For example, place the orders CSV under raw/orders/ and the products CSV under raw/products/.

4. Check the object list to confirm the files uploaded successfully.

5. Verify the orders file and the other source objects are present.

```
s3://data-engineer-sales-optimization-rakesh-2026/raw/orders/\ns3://data-engineer-sales-optimization-rakesh-2026/raw/order_
```

An early folder-placement mistake involving order_items inside customers was corrected. Before running crawlers, verify each dataset is under its own intended prefix.

5. IAM Role and Permissions

Step 6 — Configure the Glue execution role

1. Open IAM in the same AWS account.

2. Locate the role AWSGlueServiceRole-OlistSalesOptimization.

3. Confirm that the role is assigned to the Glue jobs and crawlers that need it.

4. Review attached policies and any S3 bucket policy.

5. Ensure the role has the required access to read Raw and Silver objects, write Silver and Gold outputs, use the Glue Data Catalog, and write logs.

During Gold-layer troubleshooting, the job encountered an S3 write-permission issue. The required PutObject/write access was added or corrected, after which the Gold job succeeded.

Security guidance: scope permissions to the necessary bucket and prefixes where practical. Never publish access keys, secret keys, or sensitive account information in the project repository.

6. AWS Glue Database and Raw Crawler

Step 7 — Create the Glue database

1. Open AWS Glue in eu-north-1.

2. Open Data Catalog / Databases.

3. Create or verify the database sales_optimization_db.

Step 8 — Configure and run the Raw crawler

1. Open AWS Glue → Crawlers.

2. Create or open the crawler named olist-raw-crawler.

3. Set its data source to the Raw S3 location containing the nine dataset folders.

4. Choose sales_optimization_db as the target database.

5. Assign the Glue execution role.

6. Save and run the crawler.

7. Wait for the run to complete, then inspect its status and table results.

The Raw crawler completed and the Raw datasets were cataloged. The recorded final catalog inventory included nine Raw tables.

7. Silver Layer — Nine ETL Pipelines

The Silver layer contains processed datasets written under the silver/ prefix. A separate Glue ETL job was created for each source dataset, followed by a crawler to catalog the resulting output.

Step 9 — Repeat the Silver ETL workflow for each dataset

1. Open AWS Glue Studio and create or open the job for one source dataset.

2. Use the appropriate Glue/Spark job type and the project Glue execution role.

3. Configure the job to read the corresponding Raw dataset.

4. Apply the transformation logic in that dataset's saved script.

5. Configure the output path under the matching silver/ folder.

6. Save the job and run it.

7. Review the run status and logs. Confirm successful completion.

8. Verify that output objects exist at the expected S3 prefix.

Step 10 — Run the Silver crawlers

1. Create or open the crawler for the Silver output.

2. Point it at the correct Silver S3 prefix.

3. Set the target database to sales_optimization_db.

4. Run the crawler and verify successful completion.

5. Confirm that the expected Silver table appears in the Glue Data Catalog.

Dataset	Catalog table	Output prefix
Orders	silver_orders	silver/orders/
Order items	silver_order_items	silver/order_items/
Customers	silver_customers	silver/customers/
Payments	silver_payments	silver/payments/
Reviews	silver_reviews	silver/reviews/
Products	silver_products	silver/products/

Dataset	Catalog table	Output prefix
Sellers	silver_sellers	silver/sellers/
Geolocation	silver_geolocation	silver/geolocation/
Category translation	silver_category_translation	silver/category_translation/

The products Silver output was specifically verified at s3://data-engineer-sales-optimization-rakesh-2026/silver/products/. Exact transformation rules vary by script; use the saved scripts as the source of truth when documenting column-level cleaning, casting, or filtering.

8. Gold Layer — Sales Analytics

Step 11 — Configure the Gold ETL job

1. Open AWS Glue Studio and open the job olist-silver-to-gold-sales.

2. Confirm the job uses the intended Glue execution role and database.

3. Review each from_catalog source in the script and confirm that the referenced Silver table exists in sales_optimization_db.

4. Confirm the output location is the Gold sales prefix.

5. Save the job before running it.

Step 12 — Troubleshoot the Catalog source error

A run failed with RESOURCE_NOT_FOUND_ERROR at a getCatalogSource call. The practical checks for this error are:

1. Open Glue Data Catalog → sales_optimization_db.

2. Verify every table name referenced in the Gold script, including silver_orders, silver_order_items, and silver_products, plus any other source tables used by the full script.

3. Check exact spelling, database selection, and whether the relevant Silver crawler completed.

4. Verify each catalog table points to the intended Silver S3 prefix.

5. Correct any mismatch, save, and rerun.

Step 13 — Run the Gold job and verify output

1. Run olist-silver-to-gold-sales.

2. Wait for the job run to finish and confirm Succeeded.

3. Open the Gold S3 prefix and verify that output objects were written.

4. Confirm the output is Parquet and is partitioned by order_year and order_month, as configured.

5. Review the job logs if the run fails.

```
s3://data-engineer-sales-optimization-rakesh-2026/gold/sales/
```

Step 14 — Catalog the Gold output

1. Create or open the Gold crawler.

2. Set its source to the Gold sales S3 prefix.

3. Set the target database to sales_optimization_db.

4. Run the crawler.

5. Verify that the sales table appears in the Data Catalog.

9. Final Glue Data Catalog Inventory

The latest recorded inventory contained 19 tables in sales_optimization_db: nine Raw tables, nine Silver tables, and one Gold table named sales.

• Raw: the nine source datasets.

• Silver: silver_orders, silver_order_items, silver_customers, silver_payments, silver_reviews, silver_products, silver_sellers, silver_geolocation, silver_category_translation.

• Gold: sales.

Refresh the Glue table list before publication if resources have changed since the last recorded check.

10. Amazon Athena Setup and Queries

Step 15 — Configure Athena query results

1. Open Amazon Athena in eu-north-1.

2. Choose the workgroup and query settings used for the project.

3. Set the query result location to the project athena-results/ prefix.

4. Confirm the query identity has permission to read the Gold data and write query results.

```
s3://data-engineer-sales-optimization-rakesh-2026/athena-results/
```

Step 16 — Query the Gold table

1. Select sales_optimization_db as the database.

2. Select or query the sales table.

3. Run a simple SELECT query to verify Athena can read the table.

4. Run the monthly aggregation and data-quality checks described below.

Monthly sales analysis — illustrative SQL

```
SELECT\n    order_year,\n    order_month,\n    SUM(item_sales_value) AS monthly_item_sales_value,\n    COUNT(DISTINCT order
```

The project query returned 24 year-month groups spanning September 2016 through September 2018. Among the recorded results, November 2017 had the highest monthly item_sales_value aggregate at approximately 1,179,143.77, while January 2018 had the highest listed order count at 7,220.

These figures are the recorded query results, not a guarantee of business meaning. item_sales_value is an item-level sales measure and should not be called profit, net revenue, or total customer payments without confirming the metric definition.

11. Data Quality Validation

Step 17 — Validate row counts and nulls

Recorded Gold validation results:

Check	Recorded result
Total rows	112,650
Rows with order_id	112,650
Rows with item_sales_value	112,650
Null order_id	0
Null item_sales_value	0
Null order_year	0
Null order_month	0

```
SELECT\n    COUNT(*) AS total_rows,\n    COUNT(order_id) AS rows_with_order_id,\n    COUNT(item_sales_value) AS rows_with_i
```

Step 18 — Check order-item uniqueness

```
SELECT\n    COUNT(*) AS total_rows,\n    COUNT(DISTINCT CONCAT(\n        CAST(order_id AS VARCHAR), '-',\n        CAST(orde
```

Recorded result: total_rows = 112,650 and distinct order-item combinations = 112,650. This validates uniqueness for the tested order_id/order_item_id combination, assuming those columns and the concatenation behave as expected. For rigorous validation, use a collision-safe key expression or group by both columns.

Step 19 — Interpret the aggregate carefully

The recorded SUM(item_sales_value) was approximately 15,843,553.24. Confirm the exact column definition and currency before publishing this number. It should not be interpreted as profit or net revenue unless the project's business logic explicitly defines it that way.

12. Architecture Diagram

Create an editable diagram in diagrams.net and export a PNG for the README. Show the source datasets, S3 Raw layer, Raw crawler, Glue Data Catalog, nine Silver ETL jobs, Silver S3 layer, Silver crawlers, Gold ETL job, Gold S3 output, Gold crawler, sales catalog table, and Athena analytics.

```
Olist CSV datasets\n        |\n        v\nAmazon S3 — Raw\n        |\n        v\nAWS Glue Raw Crawler → Glue Data Catalog\
```

Recommended files: architecture/olist-data-lake-architecture.drawio and architecture/architecture-diagram.png.

13. Troubleshooting Reference

Crawler does not discover expected tables

• Check the exact S3 source prefix.

• Check the target database.

• Confirm files are present and readable.

• Review crawler logs and table names.

• Check whether a table was created with a different name.

Glue Catalog source entity not found

• Verify database and table names in the script.

• Confirm the Silver crawler completed.

• Confirm the table exists in the catalog.

• Check the table's S3 location.

Glue cannot write to S3

• Verify the job's IAM role.

• Check required write/PutObject permissions for the output prefix.

• Review bucket policies and explicit denies.

• Confirm the output path.

Athena query fails

• Verify database and table selection.

• Check the Glue table's S3 location.

• Confirm Athena results location.

• Check S3 and catalog permissions.

Job runs for a long time

• Inspect Glue run details and CloudWatch logs.

• Check input paths and input size.

• Review worker configuration.

• Set a sensible timeout based on observed run time. A 480-minute timeout is a maximum allowance, not an expected runtime.

14. Security, Cost, and Cleanup

Step 20 — Review security

1. Keep S3 public access blocked unless intentionally and safely changed.

2. Use least-privilege IAM policies.

3. Do not commit credentials or secrets to GitHub.

4. Review bucket policies and access settings before sharing the project.

Step 21 — Review AWS costs

1. Open Billing and Cost Management.

2. Review current charges and service-level costs.

3. Check Free Tier or account-plan usage and expiry information available in the console.

4. Review Glue jobs/crawlers, S3 storage, and Athena query activity.

5. Decide which resources to retain for demonstration and which can be removed.

Do not delete the bucket, catalog, or job scripts until the project artifacts are backed up and you have decided whether you need the AWS environment for future demonstrations.

15. Final Checklist and Portfolio Packaging

■ Raw source data uploaded and verified.

■ Raw crawler completed.

■ Nine Silver ETL jobs completed.

■ Nine Silver outputs cataloged.

■ Gold ETL job succeeded.

■ Gold Parquet output verified.

■ Gold crawler created the sales table.

■ Athena monthly analysis completed.

■ Row count, null, and order-item checks completed.

■ Architecture diagram exported.

■ Documentation saved as DOCX and PDF.

■ Actual Glue scripts and SQL saved locally.

■ README prepared.

■ GitHub repository published, if desired.

■ AWS billing and retention review completed.

Suggested repository layout

```
aws-glue-sales-optimization/\n■■■ README.md\n■■■ docs/\n■   ■■■ project-documentation.md\n■■■ architecture/\n■   ■■■ a
```

Use the real script filenames and exact SQL that you saved from AWS. The filenames in this layout are suggestions.

16. Conclusion

The project demonstrates an end-to-end AWS data lake workflow: source ingestion into S3, schema discovery with Glue crawlers, transformation through Silver and Gold ETL jobs, cataloging, Athena analysis, and selected data-quality checks. The technical pipeline is recorded as working. The remaining work is primarily packaging, reproducibility, architecture presentation, and cost review.